



**Capstone Progress Report: Design and
Development of an AI Performance
Analytics Platform for Esports**

CIIC4151-001#

Prof. Wilson Rivera Gallego

Integrantes:

Diego A. Ramos Villanueva

Ebrain A. Mendoza Ortiz

Jorge A. Ortiz Ramirez

Hector U. Sotelo Caraballo

1. Introduction

a) Background and Motivation

In digital games and esports competitions, team and player performance is often evaluated subjectively, relying on experience, intuition, and perceived success during practice sessions. Unlike traditional sports, which have well-established platforms for statistical analysis and data-driven decision making, many collegiate and amateur esports teams lack accessible team-specific analytics tools.

As a result, teams frequently enter tournaments with limited insight into historical data such as pick and ban patterns, frequently used strategies, performance trends, or opponent tendencies. At the other end, some teams use community data, metas, and trends to build their strategies, but this ends up being too broad and often misses what the team specializes in. This absence of quantitative analysis forces teams to make strategic decisions without objective data, reducing their ability to optimize preparation and adapt to competitive environments.

The motivation for this project is to address this gap by developing a web-based esports analytics platform that transforms raw match data into structured insights, performance metrics, and predictive indicators. By centralizing data processing and analysis, the system aims to support more consistent, objective, and efficient decision-making for esports teams.

b) Stakeholder Justification

To validate the need for this project, both stakeholder feedback and academic research were considered.

Stakeholder interviews were conducted with collegiate esports head coaches from GTU, NIU and GWU. The responses reveal consistent reliance on manual data tracking, spreadsheet analysis, and subjective evaluation methods. The GTU Head coach reported primarily tracking win rates, compositions and ban patterns manually, noting that such tracking “serves mostly to back up what you’re feeling about a team.” This indicates that current data usage is supportive rather than influential in decisions for the team. Similarly, the GWU Head Coach described using Excel and third-party tools to model pick and ban probabilities, sometimes applying statistical methods such as binomial probability and Bayes’ theorem. Unfortunately, this process was described as time consuming and error prone. Across all interviews, a consistent theme emerged: teams depend on fragmented tools and manual workflows to prepare for competition. This demonstrates a clear need for a centralized system that automates data processing and supports structured analysis.

The need is further supported with the various academic research made in esports analytics throughout the years. Drachen et al. [9] demonstrated that player behavior could be analyzed using clustering techniques, revealing structured patterns within larger amounts of data. Their following work [10] confirmed that using multi-dimensional metrics were suitable for systematic analysis. In terms of match outcomes, Yang et al. [11] showed that early-game factors such as early game resource advantages and combat patterns were strong predictor indicators for Mobile Online Battle Arena games. Similarly, Costa et al. [12] demonstrated

that draft-phase information combined with historical performance can be used to predict match outcomes effectively.

However, while these kinds of research focus on demonstrating effectiveness of such analysis and isolated models, they do not provide integrated systems for accessible real-world use. Existing tools similarly lack support for team-specific workflows, centralized data management, and combined analytics capabilities. Therefore, both stakeholder feedback and existing literature confirm a clear gap: although data-driven methods for esports analysis are well-established, they are not readily accessible through unified platforms designed for team-level decision-making. This project addresses that gap by developing an integrated system that combines data ingestion, analytics, and visualization into a single, practical tool for competitive

2. Problem Statement

a) Problem Definition and Capstone Relevance

Esports teams, at collegiate and semiprofessional level, lack centralized and structured tools for team-oriented performance analysis and that also provides strategic preparation. While large volumes of structured data are available through official APIs, this data remains underutilized due to the absence of integrated systems capable of processing, analyzing, and presenting it in a form suitable for decision-making.

Current approaches rely heavily on manual workflows, including spreadsheets, VOD reviews, and fragmented third-party tools. These methods are time consuming, prone to human error, and they work in a limited scope. This prevents teams from performing consistent analysis of key competitive factors such as draft strategies, player performance trends, opponent tendencies, and real-time match dynamics. Because of this, strategic decisions are based on intuition rather than a systematic data driven evaluation.

Although existing platforms provide partial solutions, they are limited in functionality. Tools such as Mobalytics focus primarily on individual player improvement, while platforms like Oracle's Elixir emphasize aggregate professional statistics. Neither provides a unified system that integrates historical analysis, predictive modeling, and team-based strategic tools tailored to collegiate and semi-professional environments.

The problem addressed in this project is therefore not only the lack of analytics, but the lack of an integrated decision-support system that combines multiple layers of analysis into a single platform. Specifically, there is no accessible tool that simultaneously supports:

- Historical performance analysis using structured match data
- Role-normalized and trend-based player evaluation
- Draft-phase decision support based on pick/ban patterns
- Team composition and matchup analysis
- Predictive modeling of player performance and match outcomes
- Real-time early-game outcome prediction using timeline data

This project addresses that gap by developing a full-stack esports analytics platform that integrates data ingestion, feature engineering, and multiple machine learning models into a unified system. The platform incorporates several predictive and analytical components, including playstyle clustering, performance regression models, pre-game win prediction, and early-game outcome prediction based on live match state, achieving strong predictive performance in specific domains such as early-game analysis

b) Target Domain and Limitations

- Target domain:
 - The project targets competitive esports teams but isn't limited to just professional players and managing staff such as coaches, analysts, and managers. This project could apply to any player or managing staff aged between 15-30 with an interest in developing themselves and their talents, but our focus will mostly be those who actively participate in organized tournaments and team-based strategy development..
- Limitations:
 - The system relies on publicly available match data, which may result in limited sample sizes for certain teams or players. Predictive insights are based on historical performance and cannot fully account for external or internal variables such as player stress, roster changes, or dynamic in-game decision-making. Therefore, the platform functions as a decision-support tool rather than a definitive predictor of match outcomes.

c) Proposed Innovation

The proposed innovation is an AI-driven analytics platform that integrates data ingestion, advanced performance metrics, and predictive modeling into a unified decision-support system for competitive teams. Unlike traditional tools that focus on static statistics or individual performance, the platform combines historical analysis, role-normalized metrics, and machine learning models to generate actionable insights. It supports multiple stages of competition, including player performance evaluation, draft-phase analysis, team composition assessment, and early-game outcome prediction using timeline data.

3. Project Objectives

a) Technical Objectives

The general goal of this project is to design and implement an AI-driven esports analytics platform that ingests official match data from the Riot Games Match-V5 API, transforms it into structured performance metrics, and provides both analytical and predictive insights through an interactive web-based system.

To achieve this goal, the project will pursue the following specific objectives:

- Data Ingestion: Design and implement a backend system capable of retrieving match data from the Riot Games Match-V5 API including match details and optional timeline data, storing it in a normalized PostgreSQL database.
- Derived Performance Metric Computation: Develop an analytics engine that computes advanced performance indicators from raw match data, including KDA ratio, CS per minute, gold per minute, kill participation, damage share, and vision score per minute, ensuring accurate and consistent metric generation across all stored matches.
- Feature Engineering and Trend Analysis: Implement a feature extraction module that generates rolling window performance metrics, trend indicators and role normalized statistics to evaluate player consistency, improvement patterns and contextual performance difference.
- Machine Learning Model integration: Design and integrate multiple machine learning models that provide predictive insights, including player performance prediction, playstyle clustering, and match outcome prediction based on historical and early game indicators.
- Interactive Analytics dashboard: Develop a web-based Single-Page Application (SPA) that enables users to visualize player performance, trends, and predictive insights, and to interact with features such as match history, role-based analysis, and team-level evaluations.
- Team based and strategic analysis tools: Implement system features that support team-level decision-making, including draft-phase analysis, team composition evaluation, and matchup-based insights for competitive preparation.

b) Measurable Performance Indicators

The following SMART performance indicators will be used to evaluate the success of the project objectives:

Data Ingestion Performance Indicators

- Successfully ingest and store at least 20 ranked matches per player for a minimum of 5 distinct players.
- Achieve an API retrieval success rate of $\geq 95\%$ during ingestion testing.
- Support database storage capacity of at least 10,000 matches without performance degradation.
- Complete ingestion of 100 matches in under 2 minutes during batch testing.

Metric Computation Performance Indicators

- Compute derived performance metrics for $\geq 95\%$ of stored matches.
- Maintain metric computation time below 1 second per match.
- Validate derived metric accuracy with an error margin below 2% compared to manual verification samples.

Trend Analysis Performance Indicators

- Generate rolling averages across 5–10 match windows for all tracked metrics.
- Detect performance trends with at least 90% consistency across validation datasets.
- Produce trend summaries in under 2 seconds per user query.

System Performance Indicators

- Maintain dashboard response time below 2 seconds for standard player queries.
- Support at least 50 concurrent users during stress testing.
- Achieve system availability of at least 99% uptime during the final testing period.

4. Solution Approach

a) Proposed Solution Overview

The proposed platform is a web-based esports analytics system that transforms official League of Legends match data into structured, actionable insights for players, coaches, and analysts. The system ingests match data through the Riot Games API, stores it in a normalized relational database, and computes derived performance metrics such as KDA, CS per minute, gold per minute, kill participation, damage share, and vision score. On top of this data layer, the platform will implement an analytics engine that supports role-based performance evaluation, rolling trends analysis and historical pattern identification. The system extends beyond traditional analytics by integrating multiple machine learning models that provide predictive insights at different levels. These include player-level performance prediction, playstyle clustering, team composition analysis, and match outcome prediction. Additionally, a timeline-based model evaluates early-game conditions to estimate match outcomes during live gameplay. At the team level, the platform provides tools for draft preparation and matchup analysis, including composition archetype detection, synergy evaluation, and win probability estimation for competing team lineups [8].

b) Gap Analysis with Existing Solutions

Existing esports analytics tools do not fully address the needs of collegiate and amateur teams. Platforms such as *Mobalytics* focus on individual player improvement, while *Oracle's Elixir* provides aggregate professional statistics for research and broadcast purposes. These tools do not support team-specific workflows, structured draft preparation, or integrated analysis based on a team's own match history. Similarly, community platforms such as *u.gg* and *op.gg* provide general statistics but are not designed for structured team preparation. They lack features such as role-normalized performance tracking, team composition analysis, and predictive modeling tailored to specific rosters. As a result, teams often rely on fragmented tools, spreadsheets, and manual analysis. The proposed platform addresses this gap by integrating data ingestion, analytics, and predictive modeling into a single system designed specifically for team-level decision-making.

c) Value Proposition

The value proposition of the proposed platform is that it turns official esports match data into clear, team-ready decisions reducing guesswork and manual spreadsheet work for coaches and players. Instead of relying on “what felt good in scrims” and other intuitions, teams can quickly see what actually works for their roster: role-based performance trends, champion and composition success rates, and pick/ban impacts across multiple matches. The platform also adds value by providing opponent scouting and draft preparation tools such as likely picks/bans, matchup

outcomes, and AI-assisted predictions with explainable factors, which helps teams plan practice priorities and tournament strategies with evidence. Overall, it delivers faster preparation, more consistent coaching decisions, and measurable improvement tracking in a single centralized dashboard built specifically for competitive team environments.

d) Benefits and Limitations

The main benefit for the target audience is that performance data is organized in one place and can be easily seen. The system puts together match stats, historical trends, and strategic patterns into an easy-to-use web-based interface. This makes it easy for users to understand performance metrics that would be hard to figure out from raw data.

The solution makes things clearer and helps people make decisions based on data by showing data in charts, graphs, and analytical summaries. Teams can find out what their strengths and weaknesses are, what patterns they see in their picks and bans, and how their overall performance has changed over time. Also, using predictive models gives teams information that can help them plan their strategies, analyze their opponents, and plan their matches. This helps them make better competitive decisions instead of just going with their gut or what they've heard from others.

Because the system only uses data from the publicly available Riot Games API, it is limited by rate limits, data availability, and sample size. For teams with a short match history, it may be less reliable to draw conclusions from statistics and look for patterns. Also, predictive insights are based on how things have gone in the past and can't take into account things that happen in real time, like changes to the roster, player stress, strategic changes, or psychological factors during a game. As a result, the platform is more of a decision-support tool than a certain way to predict outcomes. Its insights should be understood in the context of the competition as a whole.

e) Required Resources

To bring the solution into operation, the system requires access to the Riot Games Developer API (Match-V5) for official match data retrieval, along with a backend hosting environment capable of running REST API services, processing analytics computations, and managing concurrent requests. A relational database management system is required to store normalized match data and computed performance metrics. The platform also requires frontend hosting to deploy the web-based dashboard and ensure responsive access for users. Additional software resources include backend and frontend development frameworks, database management tools, version control systems, and testing utilities for validation and performance evaluation. Finally, stable internet connectivity, API key management, and basic security configurations such as HTTPS, environment-based secret storage, are required to ensure secure and reliable system operation.

f) Intellectual Property

The project utilizes publicly available match data provided through the Riot Games API under Riot's developer policies; this data remains the intellectual property of Riot Games. The original intellectual property generated by this project includes the system architecture design, database schema, analytics pipeline implementation, derived metric computation logic, and predictive modeling framework including the feature extraction for rolling trends. While many of the mathematical formulas used (ratios, averages, probability concepts) are standard statistical methods, their specific integration, implementation, and aggregation within the platform constitute original software design and engineering work. The combination of data processing workflows, performance aggregation logic, and predictive modeling configuration represents proprietary system design developed as part of this project.

5. Technical Approach

a) System Architecture

The system follows a modular client-server web application architecture, consisting of a Single-Page Application (SPA) frontend and a RESTful backend service. The frontend, implemented as a browser-based dashboard, is responsible for presentation logic, user interaction, filtering, and data visualization.

The backend exposes structured HTTP endpoints that manage data ingestion from the Riot Match-V5 API, normalization and persistence in a PostgreSQL relational database, computation of derived performance metrics, and generation of aggregated analytics.

Predictive analytics are implemented as part of a feature extraction pipeline, which computes rolling performance trends and feeds them into machine learning models for training and inference.

The client and server communicate via RESTful API requests using JSON over HTTPS.

The architecture is composed of five layers:

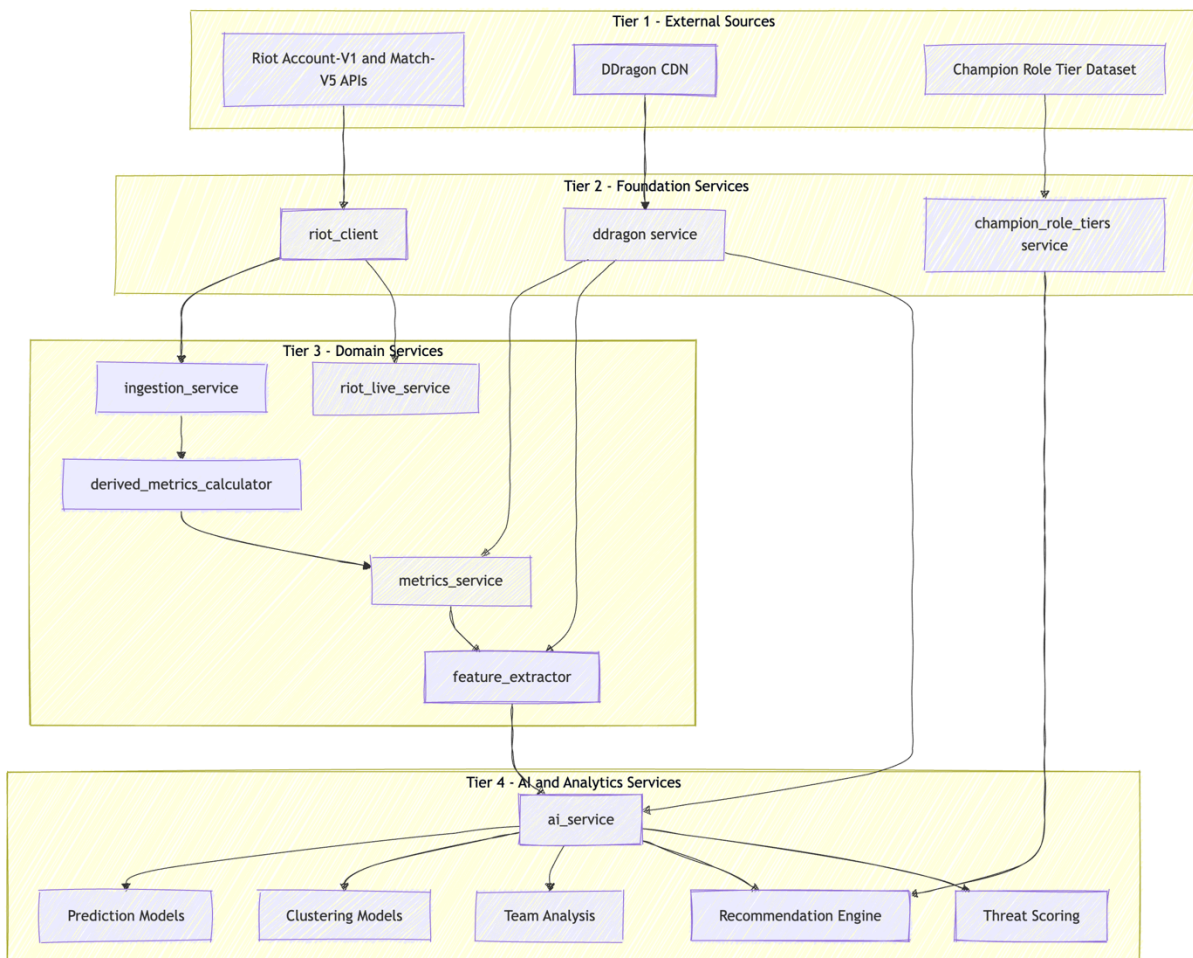
- i) **Data Ingestions:** handles API integration and rate limits concerns.
- ii) **Data Persistence:** responsible for normalized data storage and querying.
- iii) **Analytics Engine:** responsible for computing performance metrics and aggregating trends.
- iv) **Machine Learning Engine:** responsible for predictive modeling, clustering, and inference based on historical and real-time data
- v) **Presentation Layer:** responsible for delivering interactive visualizations and dashboards to the end user.

b) System Diagrams

To facilitate understanding of the system structure, data flow, and analytics logic, the project includes the following diagrams:

i. Module Interrelationship Diagram

Figure 1 presents the module interrelationship diagram for the backend system, organized into four logical tiers that illustrate how data flows through the application.



External Sources: Includes the Riot Games Account V-1 and Match V-5 APIs for match data, the Data Dragon CDN for champion and rune metadata and a locally maintained champion assessment for strategic analysis.

Foundation Services: The Riot API client manages request execution, retries, rate limits. While the DDragon service provides cached champion and rune metadata, while the role tier service provides precomputed champion rankings.

Domain Service: Implements core logic including ingestion, real-time stat retrieval, derived metric computation, aggregation, and feature extraction. This layer transforms raw data into structured analytics outputs.

AI and Analytics Service: Manages machine learning models and advanced analytics, including performance prediction, clustering, team composition analysis, threat scoring, and recommendation systems.

ii. Database Schema and ER Diagram

Figure 2 shows the Entity-Relationship Diagram of the system’s normalized PostgreSQL database, which consists of twelve tables centered around the matched entity.

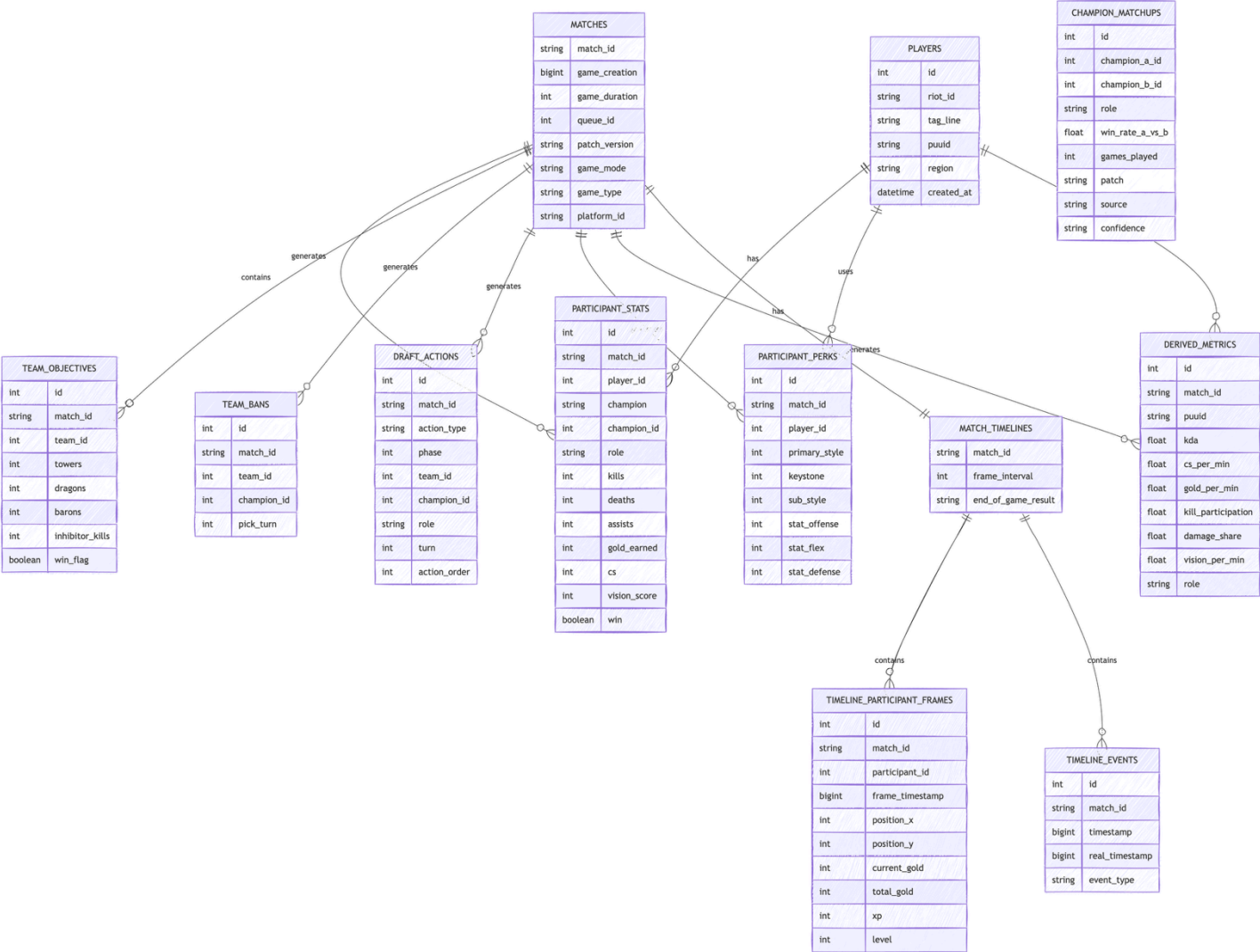
Core Entities: player, matches, participation_stats

Supporting Entities: team_objectives, team_bans, draft_actions, participant_perks

Derived Data: derived_metrics

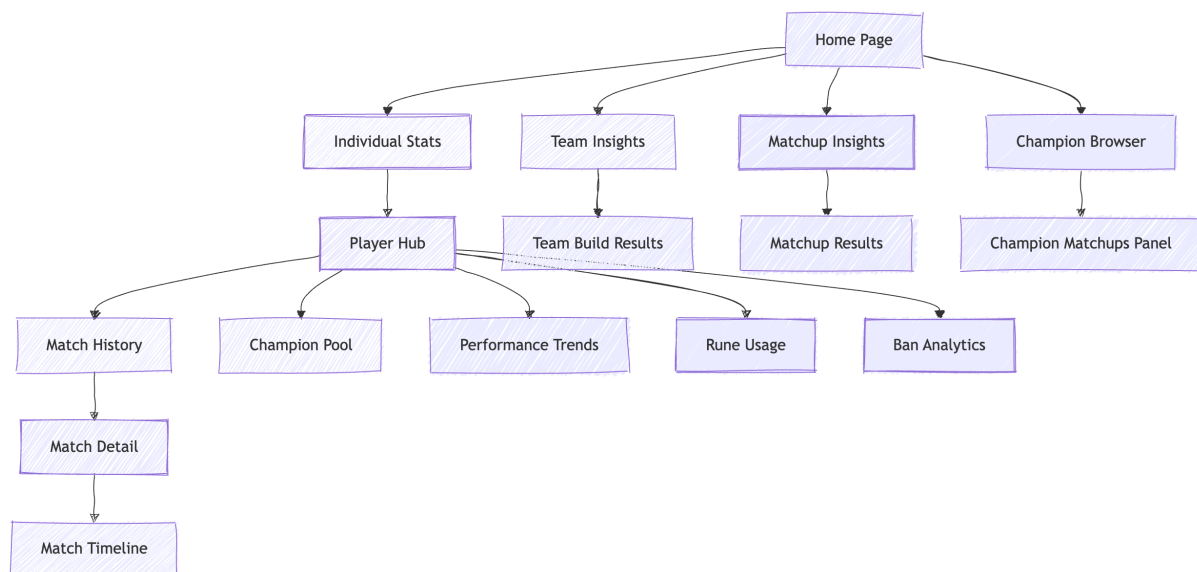
Timeline Data: match_timelines, timeline_participant_frames, timeline_events

External Reference: champion_matchups



iii. Frontend Wireframes and UI structure.

Figure 3 presents the UI wireframes and navigation structure of the frontend application, organized into four primary workflows:



Individual Player Analysis: Includes player dashboard, match history, champion pool, trends, rune usage, and ban analytics

Team Insights: Allows users to input a roster and receive composition analysis, synergy insights, threat scores, and predicted carry roles.

Matchup Insights: Compares two team compositions and provides win probability, lane-level comparisons, and matchup advantages.

Champion Browser: Displays champion statistics, tier rankings, and head-to-head matchup data.

iv. Key algorithm Flowcharts

Figure 4: Data Ingestion Pipeline represents the ingestion pipeline transforms user input into structured analytics data:

Riot ID → PUUID Resolution → Match ID Retrieval → Match Data Fetch → Database Storage → Derived Metric Computation → API Exposure → Dashboard Visualization
 The process ensures duplicate match prevention, transactional database writes, separation of raw and derived data, and optional timeline ingestion.

Figure 5: Machine Learning Pipeline: the machine learning pipeline includes training and inference stages:

- Feature extraction using rolling windows with strict temporal constraints
- Temporal train-test split without shuffling
- Model training and evaluation
- Selection and serialization of the best-performing model
- Runtime inference using the same feature pipeline

Figure 4: Data Ingestion Pipeline

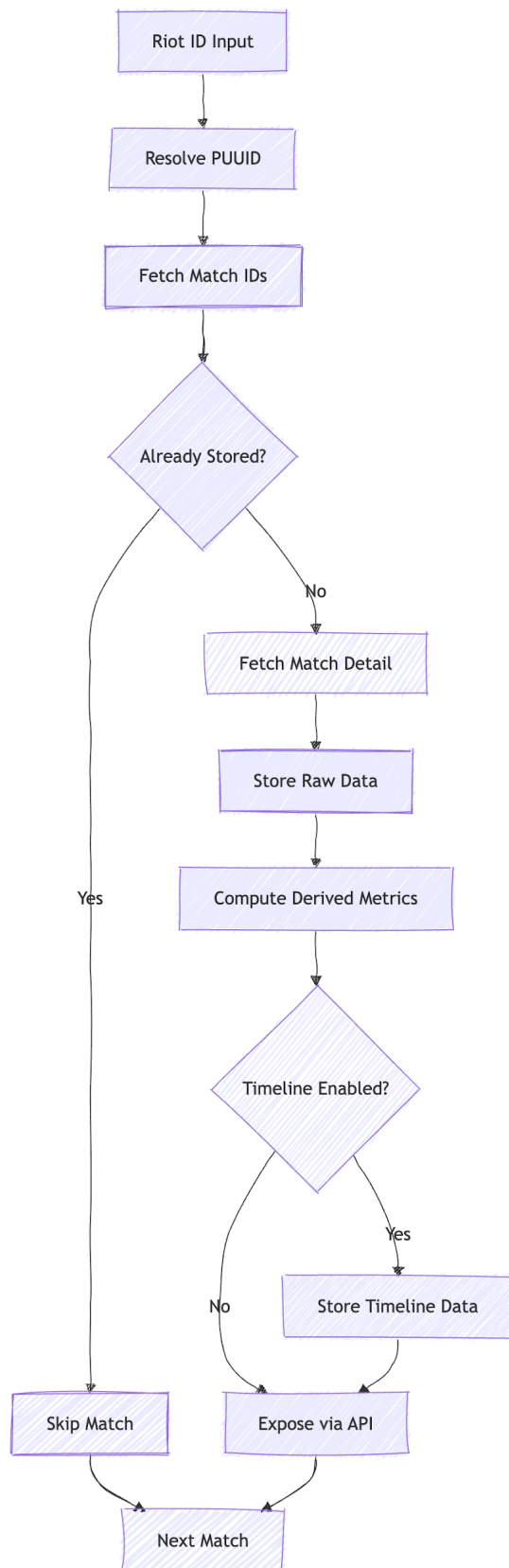


Figure 5: Machine Learning Pipeline

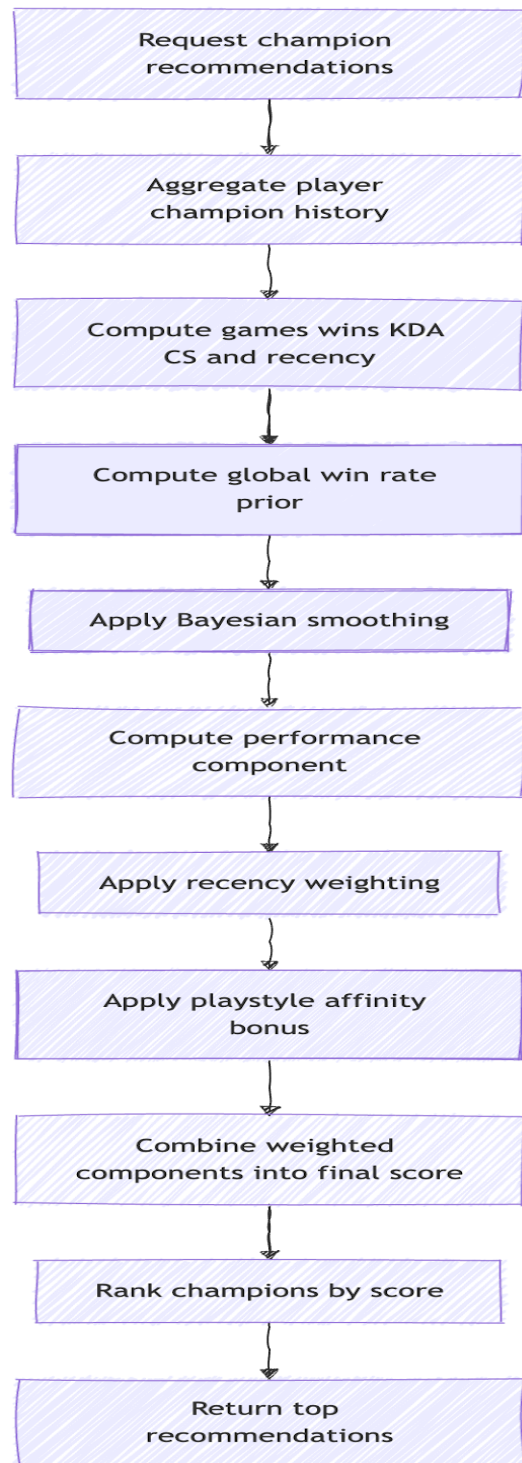
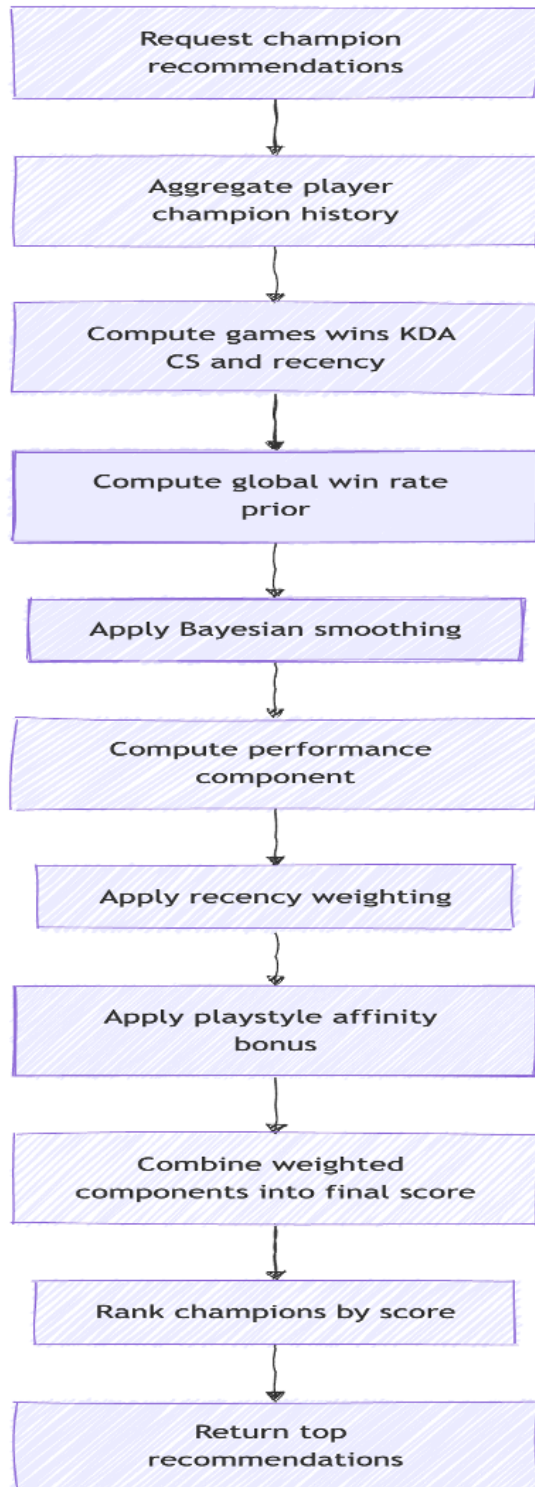


Figure 6: Champion Recommendation Algorithm



The recommendation system ranks champions using a composite score based on Bayesian-smoothed win rate,

performance metrics, recency weighting, and playstyle affinity.

c) Related Works

The proposed platform is built upon established APIs, analytics tools, and industry-standard development frameworks. The primary data source is the Riot Games Match-V5 API [1], which provides official and structured access to match metadata, participant-level statistics, team objectives, and pick/ban information for League of Legends.

Existing esports analytics platforms such as Oracle's Elixir [2] and Mobalytics [3] were reviewed as related works. Oracle's Elixir provides aggregate professional-level statistics and historical esports data, primarily for broadcast and high-level analysis. Mobalytics focuses on individual player improvement through performance summaries and overlays. While both platforms offer valuable insights, neither provides a unified workflow that integrates official API ingestion, normalized team-specific storage, customizable dashboards, structured draft analytics, and predictive modeling tailored to collegiate and semi-professional teams. This identified gap directly motivates the proposed solution.

From a technical framework perspective, the frontend is built using React, a widely adopted JavaScript library for building interactive single-page applications. The backend is

implemented using FastAPI, a high-performance Python web framework optimized for RESTful API development. PostgreSQL is used as the relational database system due to its ACID

compliance, scalability, indexing capabilities, and strong support for analytical queries. These technologies are industry-standard tools for scalable web-based analytics platforms.

i) Engineering Constraints

A primary engineering constraint is the limitation imposed by the Riot Games Match-V5 API, as well as the free tiers regarding Supabase, Vercel and Railway hosting, particularly their rate limits, availability, and dependency on a third-party service. Because match ingestion relies on external API calls, the system must implement controlled request scheduling, retry mechanisms with exponential backoff for HTTP 429 responses, and local caching of previously retrieved matches to prevent redundant calls. These constraints directly affect ingestion throughput and reliability, especially when importing multiple players or entire team histories. Additionally, match schemas may vary across patches or queue types, requiring robust validation and defensive metric computation logic that gracefully handles missing or evolving fields without breaking analytics pipelines.

Performance and scalability also represent critical constraints. The interactive dashboard requires low response times even as stored match data grows. Users will filter by player, role, champion, time window, and potentially team groupings, which increases query complexity. To meet performance requirements, the database must be properly normalized and indexed. Expensive computations such as rolling averages or win-rate aggregations may require precomputation or caching to ensure that dashboard queries remain below defined response time thresholds. The system must therefore balance analytical flexibility with query efficiency as data volume increases.

Finally, security and operational reliability are additional constraints. Riot API keys must be securely stored using environment variables and excluded from source control. If user authentication is implemented, the backend must enforce secure password storage, proper session management, and input validation to prevent common vulnerabilities. Even if the platform primarily processes publicly available match data, it must still maintain data integrity and controlled access to stored analytics. These constraints are realistic for a deployable web-based analytics platform and ensure the system remains reliable, scalable, and secure within a competitive esports environment.

ii) Engineering Standards

- REST API Design Standards

The backend follows RESTful API conventions, using resource-based endpoints, standard HTTP methods, and appropriate HTTP status codes. This standard improves interoperability between frontend and backend components and allows future integration with third party tools or services without needing to redesign the API.

- Swagger Documentation Standard

All backend endpoints are documented using Swagger. This standard enables easier frontend-backend integration, automated testing, and future maintenance. It also improves onboarding experience for new developers and supports validation of request/response schemas.

- HTTP and Web Standards

The system follows HTTP standards for authentication headers, request validation, and secure communication over HTTPS. These standards ensure reliable client-server communication and align the platform with industry best practices for web-based systems.

- Database Normalization Standards

The database schema follows relational normalization principles to eliminate redundancy and enforce referential integrity between entities such as players, matches, and derived metrics. This standard improves data consistency, reduces storage duplication, avoids updating anomalies, and improves database design, between others.

- Coding Style and Versioning Standards

The project enforces PEP8 coding standards for Python and ESLint/Prettier for JavaScript. Since code is often read more than written, to ensure readable, consistent, and maintainable code across team members.[4][5][6]

iii) Best Practices

- Version Control and Collaboration Practices

- The team uses Git with meaningful commit messages and feature-based branching. Pull requests and peer reviews are conducted for major changes to catch errors early and improve code quality. This practice minimizes integration issues and supports collaborative development.

- Testing Practices

A layered testing approach is used:

- 1) Unit tests validate analytics computations such as KDA, CS per minute, and kill participation.
- 2) Integration tests verify end-to-end functionality from data ingestion to dashboard visualization.
- 3) Regression testing ensures new features do not break previously working components.

These practices directly support compliance with project objectives and engineering constraints.

- CI/CD and Automation Practices

- Automated checks are run during development to validate builds and tests before merging code. This reduces human error, enforces consistency, and ensures the system remains stable as features are added. This will be implemented using primarily GitHub Actions. [7]
- Security Best Practices
 - Sensitive information such as Riot API keys and database credentials are managed through **environment variables**, never hardcoded into source files. Input validation and access control are applied to backend endpoints to protect competitive data and user privacy.
- Performance and Reliability Practices
 - Database indexing, query optimization, and caching of frequently accessed analytics are applied to maintain fast dashboard response times. Logging and error tracking are used to monitor API failures and ingestion issues, improving fault detection and debugging.
- Agile Development Practices
 - The project follows an **Agile-inspired sprint process** with regular planning, reviews, and retrospectives. Tasks are tracked using issue boards, ensuring transparency, accountability, and continuous refinement of both development and testing plans.

6. Project Plan

a. Development Process

The project will be developed over a three-month period (February–April) following an Agile-inspired iterative development process. The team will organize the work into two-week sprints to ensure incremental progress, early validation, and continuous refinement of both technical components and testing procedures. Each sprint will include sprint planning, implementation of prioritized features, unit and integration testing, a sprint review to evaluate progress, and a retrospective session to revise development and testing plans if necessary.

The project will be divided into three main phases. During February, the focus will be on system architecture design and data integration. This includes reviewing Riot Match-V5 API documentation, designing the relational database schema, implementing the data ingestion pipeline, and validating correct storage of match data. In addition, the team will begin preparing the feature extraction pipeline required for downstream analytics and machine learning models. At the end of this phase, a system integration review will confirm that match data can be reliably retrieved and stored and prepared for analysis.

During March, the focus will shift to implementing the analytics engine and machine learning components. This includes computing derived performance metrics such as KDA ratio, resource acquisition rate (CS per minute), gold per minute, kill participation percentage, damage share percentage, and vision score per minute. Rolling averages and trend analysis modules will also be developed and validated. In parallel, the team will implement feature extraction pipelines, train initial machine learning models, and evaluate their performance using historical match data. Performance benchmarking will be conducted to verify both metric computation and model inference efficiency.

During **April**, the team will finalize the interactive dashboard, integrate machine learning outputs into the frontend, and conduct comprehensive system validation. This phase includes stress testing, user acceptance testing, and optimization of system responsiveness. User acceptance testing will be conducted in the later stages of the month, as early development will prioritize functional validation of the user interface rather than user experience evaluation. Model inference endpoints and recommendation features will be validated within the end-to-end application workflow to ensure correctness and consistency of outputs. Final documentation, scope review, and potential expansion to include additional models for deployment configuration will also be completed, along with system packaging for delivery. This phased and iterative approach ensures systematic development, measurable validation at each stage, and continuous refinement of both analytics and machine learning components.

b. Timeline and Milestones

February – Data Foundation and Backend Setup -[COMPLETED]

Milestones: Completion of Riot API integration; finalized database schema; successful ingestion of at least 100 matches. Initial feature extraction pipeline.

Deliverables: Functional ingestion module; database schema documentation; API validation report; Feature extraction module.

March – Analytics Engine Implementation -[COMPLETED] - In review for expansion]

Milestones: Implementation of all derived performance metrics; operational trend analysis module; stable backend-database integration. Stable backend database and ML integration.

Deliverables: Metric computation engine; trend analysis module; Machine learning models and evaluation results; integration testing report.

April-May – System Validation and Deployment-[In progress]

Milestones: Fully operational dashboard; completion of stress testing; completion of user acceptance testing; final system deployment; Fully operational dashboard with integrated analytics and AI outputs.

Deliverables: Operational analytics dashboard; stress testing performance report; user acceptance testing summary; final documentation and source code repository.

c. Team Responsibilities

In response to concerns regarding project complexity and task distribution across team members, our system scope was expanded beyond the initial analytics platform to include multiple independent yet integrated subsystems. These additions include a machine learning pipeline with multiple predictive models (classification, regression, clustering, and early-game prediction), a feature extraction and trend analysis engine, team composition and matchup analysis modules, champion recommendation systems, and full end-to-end API integration with deployment infrastructure.

Each of these components introduces distinct technical challenges, including data engineering, statistical modeling, real-time inference, system integration, and frontend visualization. As a result, the project now consists of clearly separable functional subsystems that require dedicated design, implementation, and validation efforts.

Machine Learning, Data Engineering & Backend Systems - **Ebrain A. Mendoza Ortiz**

Responsible for the core system architecture and all backend-intensive components, including:

- Development of the complete data ingestion pipeline using Riot APIs (Account-V1 and Match-V5), including PUUID resolution and match retrieval
- Implementation of derived metrics computation and feature extraction pipelines (rolling averages, trend features)
- Design, training, and evaluation of multiple machine learning models:
 - Win prediction (classification)
 - Performance regression (KDA, CS/min)
 - Early-game outcome prediction (timeline-based)
 - Player and champion clustering models
- Development of AI inference endpoints and integration into the REST API layer
- Implementation of advanced analytics endpoints, including champion analytics and recommendation systems
- Deployment configuration (Railway, Supabase, Vercel) and backend optimization
- System-wide testing, including unit, integration, regression, and stress testing coordination

Analytics Standards & Champion Data Systems - **Hector U. Sotelo Caraballo**

Responsible for ensuring analytical consistency and domain-specific data correctness, including:

- Development and validation of champion-related endpoints and analytics
- Standardization of statistical outputs and metric definitions across the platform
- Validation of data accuracy and consistency for champion-level and gameplay-related analytics
- Support for integration of champion data into higher-level analytics workflows

Frontend System Design & MVP Dashboard - **Diego A. Ramos Villanueva**

Responsible for the design and implementation of the user-facing application, including:

- Development of the initial MVP dashboard interface and overall UI/UX structure
- Implementation of frontend architecture using a Single-Page Application model
- Design of user workflows for player insights, team analysis, and champion exploration
- Integration of frontend components with backend endpoints for data visualization
- Development of interactive views including performance summaries, match history, and analytics displays

API Integration & System Connectivity - **Jorge A. Ortiz Ramirez**

Responsible for system integration and ensuring communication between components, including:

- Implementation and validation of API endpoint connections between frontend and backend
 - Handling of request/response flows and data synchronization across system layers
 - Support for deployment connectivity and environment configuration
 - Contribution to system integration testing and validation of end-to-end workflows
- d. Testing Strategy

Testing will be conducted continuously throughout the development lifecycle to ensure compliance with functional objectives, scalability targets, availability requirements, and performance constraints. Unit testing will validate individual components including API response parsing, derived metric calculations, database write and retrieval operations, and trend analysis functions. A minimum pass rate of 95% will be required, and metric validation will maintain an error margin below 2% when compared to manual verification samples.

Integration testing will verify end-to-end system functionality, ensuring correct data flow from Riot API ingestion to database storage, metric computation, and dashboard visualization. The system must successfully process at least 95% of ingested matches without data loss.

Regression testing will be performed after each sprint to ensure that newly implemented features do not break previously validated functionality. Automated test suites will be executed regularly to maintain system stability.

Stress testing will simulate at least 50 concurrent users and high-volume match queries to validate load tolerance. The system must maintain dashboard response times below 2 seconds and demonstrate stability without crashes under simulated load conditions.

User acceptance testing will involve at least three League of Legends players evaluating dashboard usability, clarity of metrics, and accuracy of trend analysis. A minimum satisfaction rate of 85% will be required.

This structured and incremental testing strategy ensures compliance with defined SMART objectives and guarantees adherence to accessibility, availability, load tolerance, and performance engineering constraints

7. Progress Report

This section presents the performance results, operational capabilities, and validation of the testing plan for the implemented esports analytics platform. The results demonstrate that the system meets its functional objectives at the current stage of development and operates reliably under expected workloads.

A. Partial performance results and documentation.

The system's performance was evaluated across three dimensions: machine learning model accuracy, analytical computation correctness, and system responsiveness.

Machine Learning Performance:

The platform implements seven machine learning models trained on historical ranked match data using a temporal split to prevent data leakage between training and evaluation sets. Model results show varying predictive strength depending on the type of problem being addressed.

The Win Predictor frames the problem as a binary classification task, meaning that the system predicts match outcome based on a player's rolling performance history. For the platform we used Logistic Regression, a linear model that estimates win probability from weighted input features, and XGBoost, a gradient-boosted decision tree that captures non-linear relationships. Both models were trained using a temporal split, with **9,652 training samples** and **2,413 testing samples**, ensuring that future data was not used during training. Model selection was based on AUC-ROC, which measures the model's ability to distinguish between winning and losing outcomes across all thresholds (where 0.5 represents random performance). We ultimately chose XGBoost as the determining model as it reaches an AUC-ROC of 0.4976 and accuracy of 50.15%, which was higher than the other model. The results for this model are effectively equivalent to random guessing, indicating it cannot be a reliable source of prediction yet. This is expected as match outcomes are heavily influenced by in-game decision making.

Feature importance analysis from the XGBoost model shows that the most influential inputs include **blue side advantage**, **win rate difference between teams**, and **recent performance metrics** such as CS/min and KDA. While these features contribute some signal, they are insufficient to overcome the inherent variability of match outcomes. Overall, this result confirms that pre-game statistical models have limited predictive capability for match outcomes, reinforcing the importance of real-time and team-relative analysis in esports decision support.

KDA and CS/min Regression Models

The KDA and CS/min models frame performance prediction as a regression problem, where the system estimates the numeric value a player will achieve in their next match. Two algorithms were evaluated: Ridge Regression, a linear model with regularization, and XGBoost Regressor, which captures non-linear relationships without requiring feature scaling. Model selection was based on R^2 , which measures how much of the variation in the target variable is explained by the model.

The **CS/min model** achieved the strongest performance, with an **R^2 of 0.8674**, **RMSE of 1.059**, and **MAE of 0.804**. This means the model explains approximately 86.7% of the variation in farming performance, and its predictions are, on average, less than one CS/min away from the true value. This confirms that CS/min is a highly consistent and predictable metric based on historical player behavior. In contrast, the **KDA model** achieved an **R^2 of 0.0268**, **RMSE of 4.115**, and **MAE of 2.762**, indicating very low predictive power. The model explains only about 2.7% of the variation in KDA, and prediction errors are relatively large. This suggests that combat performance is highly variable and influenced by in-game factors such as team fights, positioning, and coordination, rather than prior statistical trends.

These results highlight that stable, repeatable behaviors such as farming are well-suited for predictive modeling, while high-variance outcomes such as combat performance remain difficult to predict.

Early-Game Predictor

The early-game predictor uses **in-game state features** (gold, XP, CS, and level differentials at 10 and 15 minutes, plus early objectives) to estimate match outcome. Logistic Regression was selected for interpretability. The model achieved an **AUC-ROC of 0.8345** and **accuracy of 75.61%** on **701 test samples** (trained on 2,804). An AUC above 0.8 indicates strong discriminative ability, meaning the model can reliably distinguish winning vs. losing game states based on early conditions. Coefficient analysis shows that **15-minute gold differential (coef $\approx$ 1.44)** and **15-minute XP differential (coef $\approx$ 0.71)** are the strongest positive predictors, confirming that mid-early resource advantages are the most influential signals. First-objective features (e.g., first dragon) contribute additional but smaller effects. Overall, this model demonstrates that **real-time game state is highly predictive**, significantly outperforming pre-game models.

Playstyle and Champion Clustering

KMeans clustering ($k = 4$) was applied to **681 players** using aggregated performance metrics. The model achieved a **silhouette score of 0.255**, indicating moderate but meaningful cluster separation.

Clusters were labeled based on centroid characteristics:

- **Support/Utility:** high vision ($\approx$ 2.98/min), high assists ($\approx$ 12.97), low CS ($\approx$ 1.79)
- **Skirmisher:** balanced damage and mobility (KDA $\approx$ 4.07, CS $\approx$ 7.52)
- **Farm Efficiency:** high CS ($\approx$ 7.31) with lower combat impact
- **Carry:** highest gold/min ($\approx$ 478.8), kills ($\approx$ 7.13), and damage share

These clusters provide **behavioral profiles** used for player analysis and recommendations, rather than predictive accuracy.

Champion Clustering

KMeans clustering was also applied to **172 champions** using aggregated stat profiles, with $k = 4$. The model achieved a **silhouette score of 0.3529**, indicating clearer separation than player clustering.

Clusters include:

- **Farm Carry**
- **Utility**
- **Versatile**
- Additional general cluster grouping

This structure supports **champion archetype identification** and enhances recommendation and matchup analysis features.

Matchup Predictor

The matchup predictor estimates match outcome by comparing **team-level aggregated features** between two teams. XGBoost was selected as the final model based on AUC-ROC. The model achieved an **AUC-ROC of 0.4898** and **accuracy of 50.51%** on **487 test samples**, indicating performance near random. Logistic Regression performed similarly (AUC = 0.4812).

Top contributing features include:

- team tracking indicators
- patch version
- role-normalized KDA
- vision and gold metrics

Despite incorporating both teams, the model remains limited by unobserved factors such as draft synergy and in-game coordination. This result reinforces that pre-game team statistics alone are insufficient for reliable outcome prediction.

B. Partial operational demonstration of the system or algorithm

The system was executed end-to-end across all major workflows, demonstrating successful integration of data ingestion, analytics processing, machine learning inference, and frontend visualization. The system successfully ingests player data through the **POST /ingest/player** endpoint, which accepts a Riot ID and tag line as input. During execution, the backend resolves the player's PUUID via the Riot Account-V1 API, retrieves their ranked match history through the Match-V5 API, and stores the data across 12 normalized database tables. All six derived performance metrics are computed at ingestion time. The pipeline includes rate-limit handling using exponential backoff for HTTP 429 responses and prevents duplicate match insertion.

Player Analytics Dashboard

Once a player is ingested, their profile page (**/player/{puuid}**) dynamically displays a six-metric KPI summary, including KDA, CS/min, gold/min, kill participation, damage share, and vision score. These values are aggregated from the `derived_metrics` table in real time. Additional views successfully render:

- rolling performance trends across recent matches
- per-champion performance breakdowns
- rune usage patterns
- ban history
- complete match logs with win/loss indicators

This demonstrates that the analytics pipeline and database queries are functioning correctly and are fully integrated with the frontend.

Match Detail and Draft Reconstruction

The system correctly reconstructs full match details through the **/match/{match_id}** endpoint. Each match view displays:

- complete 10-player scoreboard
- team objectives (first blood, towers, dragons, barons)
- full pick/ban draft sequence

Timeline data, when available, provides per-frame gold, XP, level, and CS progression, enabling early-game analysis. This confirms successful ingestion and storage of both match and timeline data.

Team Composition and Matchup Prediction

The system successfully executes team-level analytics workflows through the **/team-insights** and **/matchup-insights** interfaces.

- The **POST /teams/build** endpoint computes team composition metrics, role coverage, and player-level threat scores.
- The **POST /teams/matchup** endpoint applies the trained XGBoost model to estimate win probability between two team compositions.

Outputs include:

- team-level aggregated metrics
- role-based comparisons
- predicted win probabilities

This demonstrates correct integration of machine learning inference within the application workflow.

Champion and Ban Analytics

The **/champions** page provides a fully functional browser of champion statistics, including metadata from Data Dragon, clustering-based archetypes, and aggregate performance metrics. The ban analytics functionality correctly identifies:

- globally most banned champions
- player-specific ban patterns

These features confirm that aggregated analytics queries are operating correctly over stored match data.

AI Inference Features

The system exposes multiple machine learning inference capabilities through dedicated endpoints. During execution, the system successfully generates:

- win probability predictions
- expected KDA and CS/min values
- playstyle classification using KMeans clustering
- champion recommendations based on Bayesian-smoothed win rates

These outputs are integrated into the frontend and used to support decision-making features, demonstrating successful deployment of all trained models.

C. Partial successful execution of your testing plan.

Testing was conducted across five categories from the originally proposed testing plan. Unit testing, integration testing, regression testing and stress testing were all performed and their results were recorded. User acceptance testing was not conducted at this stage due to the frontend still being under active development. This testing will be completed in subsequent iterations once the UI reaches stability

Unit and Integration Testing

Unit and integration tests were implemented using pytest and executed against a local SQLite in-memory database seeded with fixture data, ensuring reproducibility without dependency on the live Supabase instance.

A total of 123 test cases were developed across the champions, teams, players, analytics, and AI/ML route modules. Of these, 112 passed and 11 were marked as expected failures (xfail). These 11 cases involve PostgreSQL-specific functionality (type casting, window functions, and array operations) that is not supported by SQLite but were verified to execute correctly on the production Supabase database.

The effective pass rate for executable tests is **100%**, exceeding the proposal's minimum threshold of 95%.

Integration testing confirmed correct end-to-end execution of the system pipeline:

- Riot API ingestion
- database storage
- derived metric computation
- REST endpoint responses

Metric completeness was validated using the **GET /backfill/status** endpoint, which reported **36,700 of 36,880 participant-match records** with fully computed metrics, achieving **99.51% coverage**, exceeding the 95% target.

Regression Testing

The full pytest suite was re-executed after each development sprint and following all bug fixes to ensure that new functionality did not break existing features.

Three defects were identified and resolved during regression testing:

- Missing validation on the **GET /champions** role filter, where invalid roles returned a 200 response instead of 422
- Similar validation gap in **GET /champions/by-role/{role}**, where abbreviations were incorrectly accepted
- Filtering logic error excluding secondary-role champions when tier data was incomplete

All issues were detected through automated tests and resolved prior to final deployment, confirming the effectiveness of the regression testing process.

Stress testing

Stress testing was conducted using Locust against the locally deployed backend connected to the remote Supabase free-tier database. The test simulated 50 concurrent users ramped at a spawn rate of 5 users per second over a 2-minute sustained window, with task weights reflecting realistic dashboard usage patterns.

Metric	Measured Value	Proposal Target
Concurrent Users	50	≥ 50
Total Requests	98	—
Failure Count	1	—
Failure Rate	1.02%	< 1%
Median Response Time (p50)	32000 ms	< 2000 ms
75th Percentile (p75)	53000 ms	—
90th Percentile (p90)	77000 ms	—
95th Percentile (p95)	82000 ms	< 2000 ms
99th Percentile (p99)	85000 ms	—
Maximum Response Time	85000 ms	—

Endpoint	Request s	Failures	p50 (ms)	p95 (ms)	Notes
GET /analytics/bans/most-banned	3	0	19000	111000	DB-bound aggregation
GET /analytics/champion/{id}/ban-rate	2	0	16069	99000	DB-bound aggregation
GET /analytics/player/{puuid}/bans	5	0	18000	101000	DB-bound aggregation
GET /analytics/player/{puuid}/role-performance	5	1	13000	19000	1 ConnectionResetError — Supabase statement timeout
GET /analytics/player/{puuid}/trends	1	0	19828	20000	DB-bound aggregation
GET /champions	11	0	5600	109000	First request cold-start; subsequent hits in-process cache
GET /health	3	0	8500	12000	No DB — high latency due to thread-pool starvation
GET /metrics/player/{puuid}	6	0	19000	20000	DB-bound aggregation
GET /players/	4	0	19000	113000	DB-bound aggregation

The stress testing results indicate that while the system remains functionally stable under concurrent load, it does **not yet meet the performance targets defined in the proposal**.

Response times significantly exceeded the 2-second threshold, particularly for database-intensive endpoints involving aggregation queries.

This performance degradation is primarily attributed to:

- use of a remote Supabase free-tier database, which introduces network latency and query execution limits
- lack of query optimization and caching for aggregation-heavy endpoints
- absence of precomputed analytics tables for frequently requested metrics

Despite these limitations, the system maintained a low failure rate ($\approx 1.02\%$) and continued to respond to requests without crashing, confirming functional scalability but limited performance scalability.

Future improvements will include:

- database indexing and query optimization
- introduction of caching layers
- precomputation of heavy analytics queries
- deployment to a higher-performance database tier

7. References:

- [1] Riot Games, “Riot Games Developer Portal – League of Legends API Documentation,” *Riot Developer Portal*. Accessed: Feb. 17, 2026. [Online]. Available: <https://developer.riotgames.com/docs/lol>
- [2] Mobalytics, “The All-in-One Companion for Every Gamer,” *Mobalytics*. Accessed: Feb. 17, 2026. [Online]. Available: <https://mobalytics.gg/>
- [3] Oracle’s Elixir, “League of Legends esports stats,” *Oracle’s Elixir*. Accessed: Feb. 17, 2026. [Online]. Available: <https://oracleselixir.com/>
- [4] G. van Rossum et al., “PEP 8 – Style Guide for Python Code,” Python Software Foundation, 2001. [Online]. Available: <https://peps.python.org/pep-0008/>
- [5] ESLint, “ESLint Documentation,” OpenJS Foundation. [Online]. Available: <https://eslint.org/docs/latest/>
- [6] Prettier, “Prettier – Opinionated Code Formatter.” [Online]. Available: <https://prettier.io/docs/en/>
- [7] GitHub, “GitHub Actions Documentation - GitHub Docs,” *docs.github.com*, 2024. <https://docs.github.com/en/actions>
- [8] L. M. Costa, R. G. Mantovani, F. C. M. Souza, and G. Xexéo, “Feature Analysis to League of Legends Victory Prediction on the Picks and Bans Phase,” in *Proc. 2021 IEEE Conf. on Games (CoG)*, 2021, pp. 1–5.
- [9] A. Drachen, R. Sifa, C. Bauckhage, and C. Thureau, “Guns, Swords and Data: Clustering of Player Behavior in Computer Games in the Wild,” *Proc. IEEE Conf. on Computational Intelligence and Games (CIG)*, 2012, doi: 10.1109/CIG.2012.6374152.
- [10] A. Drachen, C. Thureau, R. Sifa, and C. Bauckhage, “A Comparison of Methods for Player Clustering via Behavioral Telemetry,” *arXiv preprint arXiv:1407.3950*, 2014.
- [11] P. Yang, B. E. Harrison, and D. L. Roberts, “Identifying Patterns in Combat that are Predictive of Success in MOBA Games,” *Proc. Foundations of Digital Games (FDG)*, 2014.
- [12] L. M. Costa, R. G. Mantovani, F. C. M. Souza, and G. Xexéo, “Feature Analysis to League of Legends Victory Prediction on the Picks and Bans Phase,” *Proc. IEEE Conf. on Games (CoG)*, 2021, doi: 10.1109/CoG52621.2021.9619019.

8. Appendix:

Project web page: <https://capstone-esports.vercel.app>

Project backend page: <https://capstone-esports-production-5631.up.railway.app>

Project repository: <https://github.com/ebrain-mendoza121/capstone-esports.git>

Environment variables:

DATABASE_URL	postgresql+psycopg://postgres.aafpbgiofdftyjpncgpz:HghasqJw7BJQWKTg@aws-0-us-west-2.pooler.supabase.com:5432/postgres?sslmode=require
PRISMA_DATABASE_URL	postgresql://postgres.aafpbgiofdftyjpncgpz:HghasqJw7BJQWKTg@aws-0-us-west-2.pooler.supabase.com:5432/postgres?sslmode=require
CORS_ORIGINS	http://localhost:3000 , http://localhost:5173 , https://capstone-esports.vercel.app
RIOT_API_KEY	RGAPI-4eac67aa-025c-457e-9650-06febe9a2d83